

13. 从极坐标到地图坐标，再到 3D 污染物范围

前面第 11 章和第 12 章已经讲过：LiDAR 先测到一条条回波曲线，经过背景扣除、距离平方校正、Klett/Fernald 反演、湿度修正和 PM 标定之后，才能得到消光、PM2.5 / PM10 和热点。

但软件系统不能停在这里。算法只能回答：

哪一个距离格子、哪一个扫描方向上 PM 高。

真正的巡查软件必须继续回答：

这个高值在地图上的哪里？离地多高？属于哪个工地、道路、厂界或投诉点？它是一条走航轨迹上的瞬时高值，还是一团可以持续追踪的 3D 污染物？

所以第 13 章的重点不是再发明一个反演算法，而是把第 12 章的算法产品放回真实空间。

第 12 章输出：azimuth=40 deg, elevation=10 deg, range=120 m, PM2.5=82

第 13 章转换：东向 76 m、北向 90 m、离地约 39 m

第 14 章展示：地图上一个半透明橙红色污染体，点击可看统计和告警

一句话总结：

第 12 章解决“这里脏不脏”，第 13 章解决“这里到底是哪儿”。

13.1 软件真正需要的空间产品是什么

先把最终目标说清楚。面向城市巡逻车、固定站和工地扬尘检测，软件最少需要 4 类空间产品：

产品	数据形态	典型 UI	解决的问题
走航轨迹色带	车辆轨迹点 + PM 值	地图上的彩色路线	哪条路、哪个工地门口出现高值
PPI 平面热力图	方位角 × 距离的 PM 网格	地图上的扇形/圆形热力图	污染热点在设备哪个方向、离多远
RHI 垂直剖面	地距 × 高度的 PM 网格	距离-高度热力图	粉尘是贴地扩散还是抬升逸散

产品	数据形态	典型 UI	解决的问题
3D 体素/污染事件	x、y、z、PM + 连通域	半透明污染团 + 告警列表	污染团范围、体积、峰值、是否超标

这里有一个很重要的工程观念：

UI 不应该直接吃原始回波数组，也不应该每个页面各算各的坐标。算法层应该统一产出 L3 空间产品快照，地图、3D、告警和回放都消费同一份快照。

这样才能保证同一个热点在地图、3D 视图、告警列表和历史回放里的位置一致。

13.2 LiDAR 天然输出的是极坐标，不是经纬度

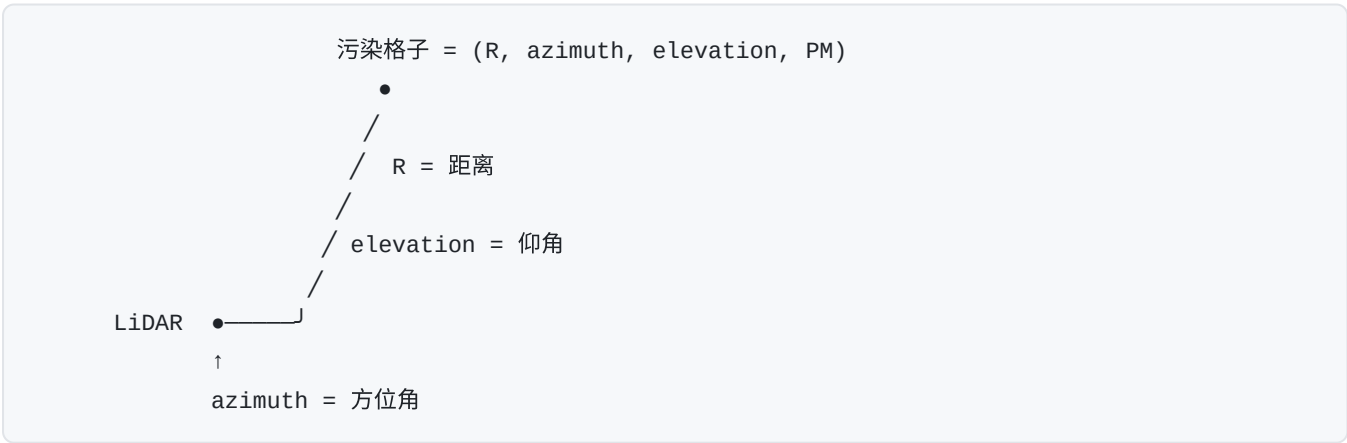
激光雷达最直接知道的是三件事：

- 1. 距离 R：回波从多远的地方回来。
- 2. 方位角 azimuth：这束光朝哪个水平方向打出去。
- 3. 仰角 elevation：这束光往上抬了多少角度。

这三个量合起来，就是一个极坐标点。

 LiDAR 极坐标示意图

可以把它想成：



所以 `pm25[120] = 82` 不是地图点。它只说明：

沿着当前这束激光的方向，在第 120 m 附近有一个 PM2.5 高值。

要让它进入地图和 3D，可视化前必须把这个极坐标点转换成真实空间坐标。

13.3 固定式系统：先转成本地 ENU 坐标

固定式 LiDAR 最常用的是 ENU 本地坐标系。它以设备为原点，在小范围内建立一个三维坐标系：

轴	英文	中文含义	软件里常用字段
E	East	东向	x_east_m
N	North	北向	y_north_m
U	Up	向上	z_up_m

 ENU 本地坐标系示意图

假设方位角定义为：

- 1. 0 度 = 正北。
- 2. 90 度 = 正东。
- 3. 180 度 = 正南。
- 4. 270 度 = 正西。

那么转换公式就是：

```
水平投影 = R * cos(elevation)

东向 x = 水平投影 * sin(azimuth)
北向 y = 水平投影 * cos(azimuth)
相对雷达高度 z = R * sin(elevation)
离地高度 height_agl = 雷达安装高度 + z
```

注意 x 用 sin、y 用 cos，是因为这里的方位角从正北开始顺时针量，不是数学课里从 x 轴开始量。

13.4 一个具体数字例子：40 度方位、10 度仰角、120 m 距离

假设某个高 PM 格子来自下面这条射线：

- 1. 方位角 azimuth = 40 deg。
- 2. 仰角 elevation = 10 deg。
- 3. 斜距 R = 120 m。
- 4. 这一格 PM2.5 = 82 µg/m³。

5. 设备安装高度 18 m。

先算水平投影和相对高度：

```
水平投影 ≈ 120 * cos(10 deg) ≈ 118.2 m
相对高度 ≈ 120 * sin(10 deg) ≈ 20.8 m
离地高度 ≈ 18 + 20.8 = 38.8 m
```

再拆成东向和北向：

```
东向 x ≈ 118.2 * sin(40 deg) ≈ 76.0 m
北向 y ≈ 118.2 * cos(40 deg) ≈ 90.5 m
```

所以这个点可以写成：

```
{
  "x_east_m": 76.0,
  "y_north_m": 90.5,
  "z_relative_lidar_m": 20.8,
  "height_agl_m": 38.8,
  "pm25_ugm3": 82.0
}
```

业务含义是：

这个高 PM 格子位于雷达东边约 76 m、北边约 90 m、离地约 39 m 的位置。

 ENU 数字换算例子

对应的 Python 函数可以很小：

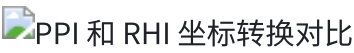
```
import numpy as np

def polar_to_enu(ranges_m, azimuth_deg, elevation_deg, install_height_m=0.0):
    az = np.deg2rad(azimuth_deg)
    el = np.deg2rad(elevation_deg)

    horizontal = ranges_m * np.cos(el)
    x_east = horizontal * np.sin(az)
    y_north = horizontal * np.cos(az)
    z_relative = ranges_m * np.sin(el)
    height_agl = install_height_m + z_relative
    return x_east, y_north, z_relative, height_agl
```

13.5 PPI 和 RHI 分别给软件什么信息

固定式扫描常见两种模式：PPI 和 RHI。



PPI：地图主屏的数据来源

PPI 是固定仰角，水平方向扫一圈。它最适合回答：

哪个方向、多远处有污染热点？

PPI 俯视图：



PPI 转 ENU 后，可以直接铺到地图上，形成：

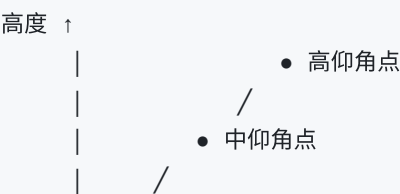
1. 工地平面 PM 热力图。
2. 设备扫描覆盖扇区。
3. 污染热点边界。
4. 污染团平面质心。
5. 叠加风向后的传播箭头。

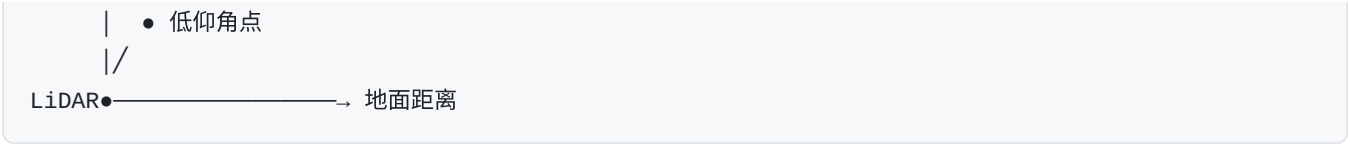
对工地、道路、厂界来说，PPI 是地图主屏最重要的数据来源。

RHI：3D 高度信息的数据来源

RHI 是固定方位角，上下扫仰角。它最适合回答：

这团粉尘抬到了多高？垂直厚度有多大？





RHI 可以先转成 “地面距离 + 离地高度”：

地面距离 = $R * \cos(\text{elevation})$

离地高度 = 雷达安装高度 + $R * \sin(\text{elevation})$

它适合做距离-高度热力图，也适合给 3D 体素补垂直结构。没有 RHI 或体积扫描时，软件也能做 2D 热力图，但 3D 污染体就只能是“拉伸式”的近似。

13.6 车载走航：坐标链比固定站多很多

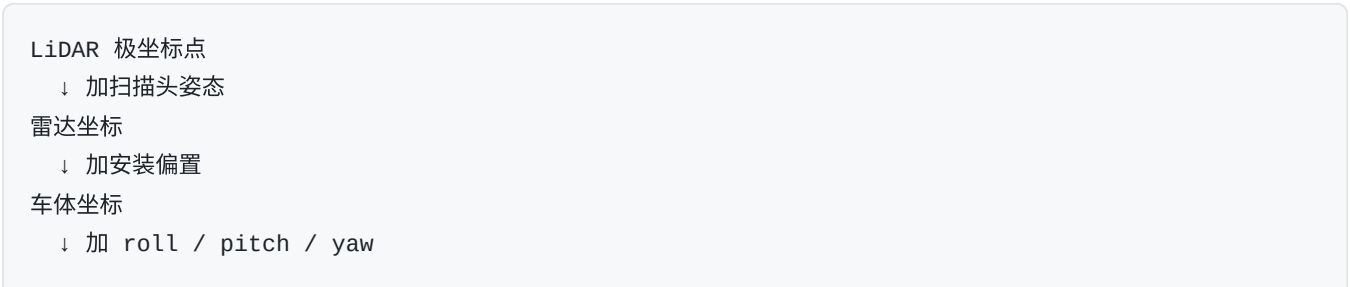
固定式系统的设备不动，坐标转换相对简单。走航车不一样：车一直在动，车体会晃，雷达还可能有安装偏置。

车载 LiDAR 坐标转换链路

车载系统每一条 profile 都应该绑定这些元数据：

元数据	字段示例	用途
时间戳	timestamp	和 GPS、IMU、扫描角度对齐
车辆位置	lon, lat, alt	把结果落到地图
航向	yaw_deg	判断车头方向
横滚/俯仰	roll_deg, pitch_deg	修正车身倾斜
雷达安装偏置	lidar_offset_xyz_m	从车体坐标转雷达坐标
扫描角度	azimuth_deg, elevation_deg	从极坐标转射线方向
车速	speed_mps	轨迹插值和质量判断

完整坐标链可以理解成：



本地 ENU 坐标
↓ 加 GPS / RTK 原点
地图经纬度

车载系统最怕的不是公式难，而是时间不同步。

车速 36 km/h = 10 m/s。如果 LiDAR 和 GPS 时间差 0.5 秒，热点位置就会错 5 m。对普通巡查显示来说只是偏一点；对污染源定位和执法取证来说，5 m 已经可能让结论不可靠。

所以走航软件必须把“时间戳、姿态、位置、扫描角度”当成数据契约，而不是最后才补的显示参数。

13.7 走航模式的两种显示数据

走航车通常有两种工作方式，UI 上也应该分开表达。

工作方式	采集策略	空间产品	UI 表现
边走边扫	车速 20-40 km/h，持续采集	轨迹点 + 沿途 PM 统计	地图上彩色路线，红色段表示高值
停车精扫	车停在投诉点或工地边界，做 PPI/RHI	PPI 热力图、RHI 剖面、体素	地图上从停车点展开扫描扇区和 3D 污染团

边走边扫适合“城市污染物巡逻”：快速发现异常路段。停车精扫适合“工地污染物检测”：在疑似源头附近做更细的空间定位。

这两种模式不冲突。最实用的业务流程往往是：

边走边扫发现高值路段
↓
系统建议停车复扫点
↓
停车做 PPI/RHI 精扫
↓
生成热点事件和证据截图

13.8 为什么一定要网格化和体素化

坐标转换之后，你会得到很多离散点：

点 1: x=76.0, y=90.5, z=20.8, PM=82
点 2: x=94.9, y=113.2, z=26.0, PM=66
点 3: x=79.0, y=87.8, z=20.8, PM=79
...

离散点对算法工程师有用，但对 UI 和业务系统还不够稳定。网格化就是把空间切成固定大小的小格子，把落在同一个格子里的点合并统计。



二维叫栅格，三维叫体素。体素就是“三维像素”。

```
i = floor((x - x0) / dx)
j = floor((y - y0) / dy)
k = floor((z - z0) / dz)
```

同一个格子里可能有多个观测值，常见统计方式如下：

统计方式	适合场景	含义
平均值	稳定显示 PM 场	多个点取平均，图更平滑
最大值	告警检测	只要格子里出现高值就突出显示
加权平均	热点质心	PM 越高，对质心影响越大
命中次数	数据质量	这个格子被扫描到多少次
最近一次值	实时刷新	保留最新观测结果

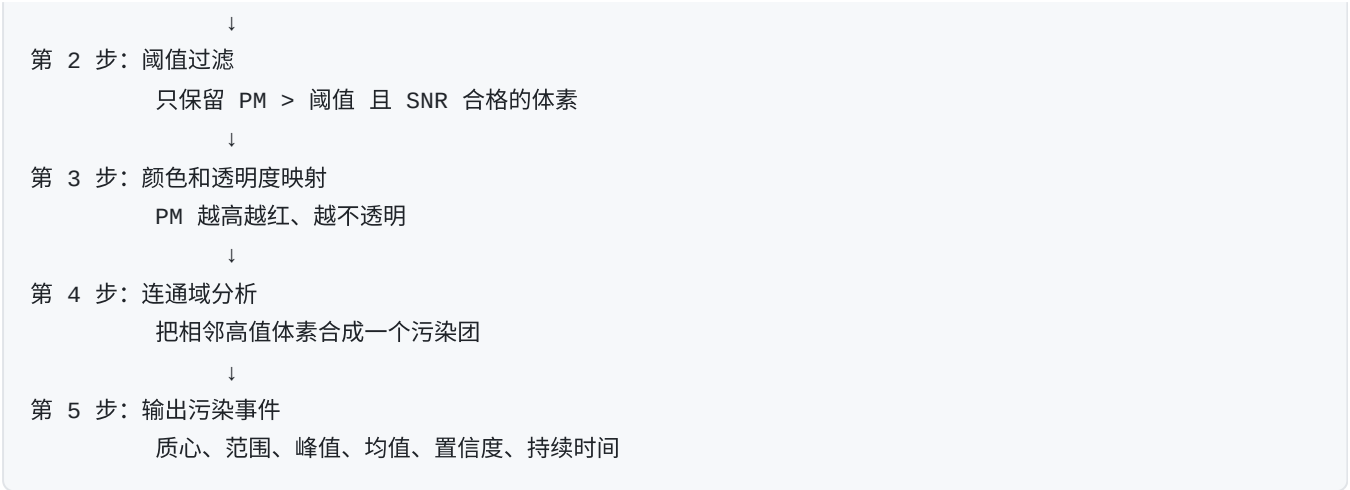
网格化不是降低精度，而是让数据变成可查询、可告警、可回放的产品。工地 PM 监测通常不需要厘米级空间精度，更重要的是稳定判断：

- 1. 哪一片区域超标。
- 2. 超标持续了多久。
- 3. 是否越过工地边界或厂界。
- 4. 处置之后有没有下降。

13.9 从体素变成 3D 污染物范围

体素网格可以直接变成地图上的 3D 污染物范围。

第 1 步：体素网格
每个体素 = (i, j, k) + PM 值 + 质量标记



过滤和着色可以这样理解：

```
import numpy as np

def voxels_to_render(voxel_pm25, voxel_snr, threshold=75):
    visible = (voxel_pm25 > threshold) & (voxel_snr >= 3)
    positions = np.argwhere(visible)

    pm_values = voxel_pm25[visible]
    alpha = np.clip((pm_values - threshold) / (pm_values.max() - threshold), 0.15, 0.85)
    return positions, pm_values, alpha
```

透明度很关键：

- 1. 低浓度边缘应该透明，让用户看见污染团范围。
- 2. 高浓度核心应该更不透明，让用户一眼看到最严重的位置。
- 3. SNR 低、湿度影响大、雨雾干扰明显的体素应该降权或不显示。

体素尺寸建议：

体素尺寸	优点	缺点	适合场景
10-20 m	渲染快、画面简洁	小热点不明显	城市巡逻总览、走航实时显示
5-8 m	细节和性能平衡	需要较稳定扫描	工地固定站默认方案
2-3 m	细节丰富	数据容易稀疏、渲染压力大	停车精扫、重点区域分析

13.10 热点事件：UI 和平台真正消费的对象

到平台层，最有价值的不是一堆散点，而是一条结构化事件。

```
{
  "event_id": "dust_20260618_102315_001",
  "timestamp_start": "2026-06-18T10:23:15+08:00",
  "timestamp_end": "2026-06-18T10:28:45+08:00",
  "mode": "fixed_ppi",
  "pollutant": "PM2.5",
  "center_enu_m": [76.0, 90.5, 20.8],
  "center_lonlat": [121.4737, 31.2304],
  "height_agl_m": 38.8,
  "azimuth_deg": 40.0,
  "range_m": 120.0,
  "peak_pm25_ugm3": 186.0,
  "mean_pm25_ugm3": 122.5,
  "mean_extinction_km_1": 0.42,
  "area_m2": 950.0,
  "vertical_extent_m": [12.0, 35.0],
  "duration_s": 330,
  "confidence": 0.91,
  "quality_flags": ["snr_ok", "humidity_corrected"],
  "exceeds_standard": true,
  "nearest_site": "工地A东北角"
}
```

这条事件能直接服务 4 个页面：

页面	怎么用事件
地图主屏	画热点标注、污染团边界和质心
3D 视图	高亮事件对应的体素连通域
告警列表	显示时间、位置、峰值、状态
回放/报表	生成处置前后对比和证据截图

这就是从“科研数组”到“业务对象”的关键变化。

```
科研数组: pm25[azimuth_index, range_index]

业务对象: 2026-06-18 10:23, 工地A东北角出现一个 PM2.5 热点,
          中心离雷达 120 m、离地约 39 m, 峰值 186 µg/m³,
          持续 5 分 30 秒, 已超标, 置信度 0.91。
```

13.11 建议的数据流：不要让 UI 自己算坐标

推荐把坐标转换放在产品层统一完成。

空间产品进入 UI 的链路

```
graph TD; L0[L0 原始回波 + 元数据] --> L1[L1 预处理产品]; L1 --> L2[L2 消光 / PM / 质量标记]; L2 --> L3[L3 空间产品快照]; L3 --> L4[地图、3D、告警、统计、回放]; L3 --> L5[统一坐标转换]; L5 --> L3; L3 --> L6[走航轨迹色带]; L3 --> L7[PPI 栅格]; L3 --> L8[RHI 剖面]; L3 --> L9[3D 体素]; L3 --> L10[热点事件];
```

L0 原始回波 + 元数据

↓

L1 预处理产品

↓

L2 消光 / PM / 质量标记

↓

统一坐标转换

↓

L3 空间产品快照

- 走航轨迹色带
- PPI 栅格
- RHI 剖面
- 3D 体素
- 热点事件

↓

地图、3D、告警、统计、回放

这样做有 3 个好处：

1. 所有 UI 页面显示同一份空间结果，不会互相打架。
2. 历史回放只需要重放 L3 快照，不必每次重新跑算法。
3. 后续接平台、报表、联动喷淋或执法系统时，数据接口更稳定。

13.12 这一章真正想让你记住什么

第 13 章可以浓缩成一条链路：

```
graph TD; A[回波距离 bin] --> B[消光 / PM 估算]; B --> C[距离 + 方位角 + 仰角]; C --> D[固定站 ENU 或车载位姿链]; D --> E[地图坐标 / 地距-高度坐标]; E --> F[栅格和体素]; F --> G[热点事件]; G --> H[地图热力图 + 3D 污染体 + 告警回放];
```

回波距离 bin

↓

消光 / PM 估算

↓

距离 + 方位角 + 仰角

↓

固定站 ENU 或车载位姿链

↓

地图坐标 / 地距-高度坐标

↓

栅格和体素

↓

热点事件

↓

地图热力图 + 3D 污染体 + 告警回放

如果只记 5 句话，就记这 5 句：

1. LiDAR 原始空间信息是“距离 + 方位角 + 仰角”，不是现成地图点。
2. 固定站先转 ENU，走航车还必须叠加 GPS、IMU、安装偏置和时间同步。
3. PPI 主要给地图热力图，RHI 主要给垂直结构和 3D 污染体。
4. 体素不是炫技，而是把离散扫描点变成可显示、可告警、可统计的数据产品。
5. 软件真正消费的不是原始回波，而是 L3 空间产品快照和热点事件。